HANOI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

INTERNSHIP TECHNICAL REPORT

BudgetRAG / MemAlign-Qwen

Resource-Aware Retrieval and Context Allocation for Grounded LLM Inference

Student: Kieu Son Tung (20235571)

Class: DSAI 02 - K68

Supervisor: Assoc. Prof. Le Thanh Huong

Department: Information Systems

Repository: https://github.com/sontungkieu/true-chat

June 5, 2026

Abstract

Long-context Retrieval-Augmented Generation (RAG) improves grounding by giving the generator more retrieved evidence, but it also increases prompt length, latency, and estimated decoder KV-cache memory. This report studies resource-aware retrieval and context allocation: for a query and a set of retrieved candidates, the system must select a retrieval-context action that preserves grounded answer quality while reducing token, latency, and estimated KV-cache costs. The implementation begins with a benchmark foundation for retriever registration, benchmark execution, and retrieval metrics. It then adds BudgetRAG context policies, deterministic adaptive profiles, generation feedback, RAGAS/MiMo answer labels, RLAIF-style answer and context labels, reward/preference construction, and offline selector baselines. The main empirical evidence is diagnostic: 192 MiMo answer labels, 192 merged MiMo context labels with 177 clean usable rows, context sufficiency rate 0.591, mean selected chunks 1.276 versus mean irrelevant chunks 3.708, and context-reward ablations that change 140 of 192 reward rows. Multi-seed held-out selector evaluation shows that shrinkage-smoothed best average is the strongest full-coverage non-oracle baseline in the current logged data, while learned selectors remain limited by data size and action coverage. The report does not claim human preference learning, online reinforcement learning, DPO/PPO/GRPO training, or deployed KV pruning. The next research step is to expand logged query groups, retriever diversity, multi-judge validation, and local Qwen/KV profiling.

Contents

1 Introduction

Retrieval-Augmented Generation (RAG) combines external retrieval with language-model generation so that answers can be grounded in retrieved evidence rather than only in parametric memory [8]. The practical baseline is often simple: retrieve the top-k documents and place most of their content into the prompt. That baseline is easy to implement, but it is not resource-aware. Retrieved context can contain irrelevant chunks, longer prompts increase latency and cost, and long sequence lengths increase decoder KV-cache memory for local LLM inference.

BudgetRAG / MemAlign-Qwen addresses this trade-off. The core question is: for a given query and a set of retrieved candidates, which retrieval strategy, context policy, and context budget should be selected to maximize grounded answer quality while controlling token count, latency, and estimated KV-cache footprint?

The MemAlign-Qwen motivation is local and privacy-constrained inference. In later phases, private documents should be processed by trusted local models rather than remote APIs. In the current phase, BudgetRAG reduces context before inference and uses RLAIF-style context labels as future evidence for local Qwen profiling and possible KV evidence masks. The current report is careful about the boundary: the system is an offline research pipeline, not a deployed production policy.

1.1 Contributions

This internship produced six concrete contributions:

1. A reproducible RAG/BudgetRAG benchmark pipeline with fixed context policies, adaptive profiles, traceable action rows, and curated reports.
2. An RLAIF-style feedback layer that records answer-level labels, context-level sufficiency labels, and direct pairwise judge audits.
3. A reward/preference construction pipeline with explicit quality, evidence-support, token, latency, KV, unsupported-claim, and error components.
4. Held-out offline selector baselines that compare cheapest, average, smoothed, learned linear, and oracle-logged policies without replacing the runtime heuristic.
5. Full MiMo context-label evidence showing that retrieved contexts often contain only a small number of useful chunks and several irrelevant chunks.
6. Analytical Qwen KV-cache estimates and a deployment-oriented vLLM model-benchmark workflow to connect BudgetRAG with future local inference experiments.

2 Problem Formulation

For a query q and a candidate set C returned by one or more retrievers, the system chooses an action:

$$a = (\text{retrieval strategy}, \text{fusion strategy}, \text{context policy}, \text{budget}, \text{adaptive profile}, \text{generator model}).$$

The objective is an offline reward that trades off answer quality and evidence support against resource costs:

$$R = w_q Q + w_s S - w_t T - w_l L - w_k K - P_{\text{unsupported}} - P_{\text{error}}.$$

Here Q is answer quality/correctness, S is evidence or context support, T is normalized token cost, L is normalized latency, K is normalized estimated KV-cache cost, and the penalty terms capture unsupported claims and failed/invalid generations or labels.

The state includes query features, retrieval diagnostics, action metadata, and cost estimates. The evaluation is **offline logged-candidate evaluation**: a selector can only choose among actions that were already logged for a query. It is not online RL, and selector artifacts explicitly keep `runtime_default_replacement=false`.

3 Related Work

3.1 RAG and RAG Evaluation

RAG introduced a practical way to combine retrieval with neural generation for knowledge-intensive tasks [8]. BEIR showed that retrieval quality varies across heterogeneous domains and that no single retriever should be treated as universally best [14]. RAGAS adds automated RAG evaluation signals such as answer relevance and faithfulness [5]. In this project, BEIR-style metrics motivate retrieval benchmarking, while RAGAS is used as an initial proxy before richer MiMo answer labels are introduced.

3.2 Prompt and Context Compression

LLMLingua, LongLLMLingua, and Selective Context study prompt/context compression for efficient LLM inference [6, 7, 10]. BudgetRAG differs in formulation: context compression is not a standalone preprocessing step, but part of a retrieval-context action evaluated under answer quality, context sufficiency, token cost, latency, and estimated KV cost.

3.3 KV-Cache Compression and Pruning

H2O, StreamingLLM, KIVI, SnapKV, and PyramidKV reduce runtime memory or attention cost through cache selection, quantization, or pruning [16, 15, 12, 11, 2]. BudgetRAG currently operates before inference by selecting a smaller or cleaner context. Runtime KV pruning is future work; context-level RLAIF labels may later provide evidence-token masks.

3.4 Graph Retrieval, RLAIF, and Contextual Bandits

GraphRAG highlights graph-based retrieval for local-to-global query-focused summarization [4]. Hybrid retrieval and Reciprocal Rank Fusion (RRF) are relevant for combining retrieval strategies [3]. RLAIF and preference learning use AI feedback to construct reward or preference signals [1]; DPO is a later preference-optimization method, not implemented here [13]. Because the BudgetRAG action is a one-step query-time decision, offline contextual bandit framing is more appropriate than PPO/GRPO in the current data-limited phase [9].

3.5 Positioning of This Work

Research line	What it solves	Remaining gap	BudgetRAG position
RAG evaluation	Measures retrieval and answer quality.	Does not choose a resource-aware context action.	Uses retrieval and answer metrics as logged feedback.
Prompt compression	Shortens prompts before generation.	Often treats compression separately from retrieval policy.	Makes compression a retrieval-context action.
KV-cache methods	Reduce runtime cache memory or attention cost.	Usually operate inside model inference.	Works before inference; KV pruning remains future work.
Graph/hybrid retrieval	Improves evidence discovery and fusion.	Needs action-level allocation evidence.	Treats retriever choice as part of future action coverage.
RLAIF/preference learning	Builds AI-feedback rewards/preferences.	General alignment methods do not target RAG context budgets.	Uses AI feedback for answer and context supervision.
Contextual bandits	Learn one-step action selection.	Require enough logged action coverage.	Evaluates offline selectors over logged candidates only.

4 System Overview

The project pipeline is:

`Query` → retriever / graph / hybrid retrieval → BudgetRAG context policy → generator → answer/context feedback → reward/preference rows → offline selector evaluation

The main implementation components are:

- `rag-bench`: benchmark, BudgetRAG, and RLAIF CLI.
- Retriever registry: BM25 plus scaffolded graph/hybrid strategies.
- Context policies: `legacy`, `char-budget`, `per-doc-budget`, `score-density`, `sentence-trim`, `evidence-aware`, and `adaptive-heuristic`.
- Adaptive profiles: conservative, balanced, and aggressive.
- RLAIF artifacts: `rlaif_actions.jsonl`, `rlaif_feedback.jsonl`, `rlaif_rewards.jsonl`, and `rlaif_preferences.jsonl`.
- Labelers: `rlaif-label-answers`, `rlaif-label-contexts`, and `rlaif-label-pairs`.
- Selectors: fixed, cheapest, best average, family-smoothed, shrinkage-smoothed, linear reward model, smoothed linear selector, and oracle logged.

4.1 Method Summary

The method can be read as three connected stages.

BudgetRAG context allocation. A query first produces candidate evidence through a registered retriever. A context action then decides how much evidence to keep and how to trim it. The action may preserve the legacy full-context behavior, impose character budgets, distribute budget per document, prefer high score-density chunks, trim at sentence boundaries, or use deterministic adaptive profiles.

RLAIF feedback and reward construction. After generation, answer-level and context-level judges label the logged row. Answer labels estimate correctness, support, unsupported claims, and refusal behavior. Context labels estimate whether the retrieved context is sufficient, which chunks are useful, and which chunks are redundant or irrelevant. These labels are converted into scalar rewards and pairwise preferences with guardrails so missing or ambiguous feedback is not treated as zero quality.

Offline selector evaluation. Selector policies are trained or summarized only on the logged train split and are evaluated on held-out query groups. The selector can choose only among candidate actions already logged for that query, which makes the experiment an offline logged-candidate study rather than an online RL deployment.

The design keeps raw experiment outputs under ignored `benchmark_results/`; committed reports are curated summaries. This prevents raw judge outputs, downloaded Kaggle working directories, or secrets from becoming part of the repository history.

5 Experimental Setup

The experiments are organized into retrieval-only, generation-feedback, and RLAIF-feedback stages. This separation keeps the evidence for each claim clear: the early phases validate benchmark plumbing and context budgeting, while the later phases evaluate answer and context quality.

5.1 Datasets and Logged Rows

The completed empirical results use SciFact-style BudgetRAG logs. Phase 1B and Phase 1C retrieval-only runs use BM25 on SciFact with generation skipped. Phase 1C.3 and Phase 1D use 192 joined action-query rows derived from BudgetRAG generation outputs and RAGAS post-hoc rows.

Item	Count
Phase 1D joined action-query rows	192
RAGAS source action files	64
Query groups in selector smoke	55
Action signatures	77
Full MiMo answer labels	192
Full MiMo context labels	192
Clean usable context labels	177
Targeted multi-judge audit rows	60

The action log is not yet broad enough to support strong retrieval-strategy allocation claims. Most completed results use BM25. Planned retriever-diversity runs add BM25, graph-BM25, and hybrid-RRF as separate logged action families.

A small sampled HotpotQA Kaggle evaluation is included later as supplementary multi-hop stress evidence. It is not part of the main Phase 1D selector training/evaluation set, and it is not reported as a full HotpotQA benchmark claim.

5.2 Models and Judges

The generation/action matrix includes MiMo, Llama 3.1 8B, and Qwen3 32B rows. RAGAS answer relevancy provides the initial proxy label. MiMo provides full answer-level and context-level labels. DeepSeek v4 Flash and Groq Qwen3 32B provide secondary labels in a targeted context audit.

The judge prompts are constrained to logged question, answer, and context. They are not allowed to browse or use external knowledge. Invalid JSON, missing answers, missing context, and ambiguous judge outputs are recorded as ambiguous/null-score rows rather than converted into zero-quality labels.

5.3 Split Rule and Selector Evaluation

Selector evaluation uses deterministic held-out splits at query level:

$$\text{split key} = \texttt{benchmark + query\_id}.$$

This prevents action-row leakage: all actions for the same query remain in the same split. Preferences that cross train/eval boundaries are dropped and counted in the split manifest. Selector costs are logged/offline normalized costs; a deployable pre-generation selector would require estimated token/KV costs and predicted latency features.

5.4 Reward Weights

The main scalar reward uses quality weight 0.75, evidence-support weight 0.10, token weight 0.05, latency weight 0.05, and KV weight 0.05, with explicit penalties for unsupported claims and errors. Context-label reward is opt-in and non-default. The reported context penalty weights are 0.25, 0.50, and 1.00.

6 Phase 1A: Benchmark Foundation

Phase 1A created the benchmark foundation: a retriever registry, benchmark CLI, retrieval metrics, and traceable output records. The metric layer includes hit@k, MRR, nDCG, latency, and context size/cost fields. This phase made later BudgetRAG experiments reproducible: fixed policies, adaptive profiles, and RLAIF builders all consume the same output format.

Phase 1A established the reproducible benchmarking infrastructure. Quantitative reporting starts from Phase 1B/1C, where committed reports contain concrete matrix outputs.

Phase	Goal	What was implemented	Main finding
1A	Reproducible RAG benchmarking	Retriever registry, benchmark CLI, retrieval metrics, traceable records.	Later phases could reuse one output schema.
1B	Fixed context budgeting	Legacy, character, per-document, score-density, sentence-trim, and evidence-aware policies.	Compression/KV savings could be measured separately from answer quality.
1C	Deterministic adaptation	Adaptive heuristic plus conservative, balanced, and aggressive profiles.	Conservative settings were stable but too cautious; profiles increased action diversity.
1C.3	Generation feedback	Multi-model generation rows, MiMo long-context runs, HotpotQA sampled Kaggle path.	Answer labels were needed; retrieval-only metrics were insufficient.
1D	RLAIF selector	Answer/context/pairwise labels, rewards/preferences, held-out selector baselines.	Context-level labels materially changed rewards and exposed evidence insufficiency.

7 Phase 1B: BudgetRAG Fixed Context Policies

Phase 1B introduced fixed context policies. `legacy` preserves the older behavior, `char-budget` clips total characters, `per-doc-budget` avoids one long document dominating the prompt, `score-density` favors high score per length, `sentence-trim` avoids cutting inside sentences, and `evidence-aware` keeps lexical/query-aware evidence spans. Importantly, `evidence-aware` is not an answer-aware entailment verifier.

The Phase 1B smoke run used 10 SciFact queries, BM25, top-k 3, skipped generation, and a Qwen2.5-14B KV profile. The 1000-character variants kept about 1000 characters with compression around 0.216 and analytical KV savings around 900 units in the report. Quality columns are blank because generation was skipped.

8 Phase 1C: Adaptive Profiles

Phase 1C replaced a purely fixed context budget with a deterministic adaptive heuristic. The selector uses retrieval diagnostics such as score gap, normalized entropy, confidence, and long-document dominance.

Phase 1C.1 expanded validation to 50 SciFact BM25 queries. The heuristic selected `evidence-aware` for 48 queries and `per-doc-budget` for 2, but still selected budget 4000 for all 50 queries. This showed stable metadata but an overly conservative policy.

Phase 1C.2 added conservative, balanced, and aggressive profiles. Balanced and aggressive profiles created more diverse budget choices. For example, with matrix budget 1000, conservative kept 4000 characters for all 50 queries, balanced averaged 2260 kept characters, and aggressive averaged 999.8 kept characters. This was still a deterministic heuristic baseline, not a learned selector.

9 Phase 1C.3: Generation Feedback Layer

Retrieval-only metrics cannot determine whether compressed context still supports a grounded answer. Phase 1C.3 added a generation feedback layer. RAGAS answer relevancy was first joined back to BudgetRAG `query_results.jsonl`; then MiMo answer labels were added for richer answer correctness, evidence support, faithfulness, and unsupported-claim penalties.

The Phase 1D selector-smoke input contained 192 joined action-query rows from 64 action files. Model

coverage was MiMo 96 rows, Llama 3.1 8B 48 rows, and Qwen3 32B 48 rows. Context-policy coverage was adaptive-heuristic 96, evidence-aware 48, and legacy 48. Budget coverage included 4000:48, 8000:48, 1000:36, 2000:36, 16000:12, and 32000:12.

The full MiMo answer-label run produced 192 labels, 192 valid JSON rows, 0 invalid JSON rows, 25 ambiguous labels, 1 judge error, and 191 scored labels. The Pearson correlation between MiMo quality and RAGAS answer relevancy was 0.277. This low correlation supports the interpretation that RAGAS answer relevancy and MiMo answer quality measure related but not identical signals.

10 Phase 1D: RLAIF Reward and Preference Pipeline

Phase 1D converts BudgetRAG logs into normalized RLAIF-style datasets:

Command	Role
`rlaif-build`	Convert BudgetRAG rows into normalized actions and feedback.
`rlaif-label-answers`	Label answer correctness, support, faithfulness, and unsupported claims.
`rlaif-label-contexts`	Label context sufficiency, useful chunks, redundant chunks, irrelevant chunks, and missing evidence.
`rlaif-reward`	Build scalar rewards and pairwise preferences with quality guardrails.
`rlaif-split`	Split by `benchmark + query_id` to avoid row-level leakage.
`rlaif-train/eval`	Train and evaluate offline selector baselines.
`rlaif-label-pairs`	Directly judge action pairs for reward calibration audits.

The reward builder preserves provenance and missing reasons. Missing, ambiguous, or invalid labels do not become score zero. Preferences are generated within comparable query groups, with guardrails that reject efficiency-only wins when answer quality drops too much.

RLAIF in this report means AI-feedback labeling for reward/preference construction. It is not full RLHF and it is not human preference learning.

11 Main Results

11.1 Headline Findings

Finding	Evidence in this report
Full context-level RLAIF labeling is now available.	192/192 MiMo context labels were merged; 177 were clean usable labels.
Retrieved contexts contain substantial noise.	Mean selected chunks 1.276 versus mean irrelevant chunks 3.708.
Context labels changed reward/preference construction.	Context penalty 0.25 changed 140/192 reward rows and increased preferences from 722 to 952.
Context sufficiency is a real bottleneck.	Sufficiency rate 0.591, with 76 insufficient contexts among the scored labels.
The strongest full-coverage non-oracle selector is smoothed, not purely learned.	Shrinkage-smoothed selector: reward 0.602, quality 0.773, coverage 1.000 in the six-seed sweep.
Multi-judge audit supports the insufficiency signal.	51/60 targeted rows were consensus-insufficient; 6 rows were high-disagreement/MiMo-harsh cases.

11.2 Full MiMo Context Labels

The first full context-label pass merged 192 action rows with no missing, unknown, duplicate, or conflicting action ids. It produced 177 clean usable labels, 15 ambiguous labels, 0 invalid JSON labels, and 1 error. The sufficiency rate was 0.591, with 110 sufficient contexts and 76 insufficient contexts.

Metric	Value
Merged labels	192
Clean usable labels	177
Ambiguous labels	15
Sufficiency rate	0.591
Mean selected chunks	1.276
Mean redundant chunks	0.286
Mean irrelevant chunks	3.708
Mean context quality	0.602
Mean evidence support	0.556
Mean minimality	0.947

The main qualitative signal is that the judge often identifies only one or two useful chunks while marking several chunks as irrelevant. This supports context-level supervision rather than answer-only reward.

11.3 Context Reward Ablation

Context labels materially changed reward/preference construction. The answer-only baseline created 722 preferences. Context penalty 0.25 created 952 preferences and changed 140 reward rows; 108 changes were negative and 32 were positive. Stronger penalties were more aggressive and compressed the reward scale.

Setting	Preferences	Changed rows	Negative deltas	Mean changed delta
answer-only	722	–	–	–
context penalty 0.25	952	140	108	-0.212
context penalty 0.50	946	140	108	-0.316
context penalty 1.00	944	140	108	-0.525

Penalty 0.25 is the conservative non-default candidate. Penalty 1.00 remains diagnostic.

11.4 Multi-Judge Targeted Audit

The targeted multi-judge audit compares MiMo labels with secondary DeepSeek and Groq labels on 60 high-impact rows. DeepSeek v4 Flash labels were available for the full 192-row context-label set and were then filtered to the same targeted subset. Groq Qwen3 32B was run directly on the 60 targeted rows. The strongest signal was 51/60 consensus-insufficient rows and 6 MiMo-harsh/high-disagreement rows. MiMo was not globally too harsh on this subset, but the disagreement rows should be inspected before increasing context penalties.

12 Qualitative Error Analysis

The aggregate metrics are best interpreted together with individual cases. This section summarizes representative rows from the local experiment artifacts; raw JSONL outputs are not included in the report repository.

12.1 Correct Abstention but Insufficient Context

Several rows receive high answer-level quality because the model correctly refuses or abstains, while the context-level label still marks the retrieval context as insufficient. The two labels measure different objects: *answer behavior* and *retrieved evidence quality*.

Query	Claim	Answer pattern	Context label
75	Active *H. pylori* urease has a polymeric structure comprising UreA and UreB.	The answer says the provided contexts do not contain the urease-structure information. MiMo answer quality/support are 1.0.	Insufficient; no selected chunks; 9 irrelevant chunks; context quality 0.0.
72	Activator-inhibitor pairs are provided dorsally by Admpchordin.	The answer says the contexts do not mention Admpchordin or dorsal activator-inhibitor pairs. Answer quality/support are 1.0.	Insufficient; no selected chunks; 8 irrelevant chunks; context quality 0.0.

These examples motivate context-level RLAIF. Answer labels alone would treat the abstention as correct, but the retriever/context layer still failed to provide evidence.

12.2 Sufficient Context with Many Irrelevant Chunks

Other rows are sufficient because one chunk contains the needed evidence, even though most retrieved chunks are irrelevant. This pattern motivates evidence-aware context allocation and future KV evidence masks.

Query	Claim	Useful evidence pattern	Irrelevant chunks
49	ADAR1 binds to Dicer to cleave pre-miRNA.	One selected chunk directly states that ADAR1 binds Dicer and promotes pre-miRNA cleavage.	8
48	A total of 1,000 people in the UK are asymptomatic carriers of vCJD infection.	One selected chunk provides prevalence data contradicting the 1,000 claim.	7
42	High microerythrocyte count raises vulnerability to severe anemia in homozygous alpha-thalassemia trait subjects.	One selected chunk shows the count is protective against severe malarial anemia, not harmful.	7

This pattern explains why the average selected chunk count is only 1.276 while the average irrelevant chunk count is 3.708. It also shows why context reduction can improve efficiency without necessarily removing evidence, provided that the policy can identify the useful chunk.

12.3 Pairwise Disagreement: Quality Tie, Efficiency Wins

The MiMo-50 pairwise audit found 7 disagreements between scalar reward-derived preferences and direct pairwise judge choices. They cluster in query 128. The scalar reward preferred a slightly higher quality/support action; the pairwise judge considered both answers acceptable, or both abstentions correct, and chose the cheaper action. This motivates `pairwise_tie_v1`: when quality/support differences fall within a small tie band, resource efficiency should carry more weight.

12.4 MiMo-Harsh / High-Disagreement Rows

The multi-judge audit identified 6 rows where MiMo marked context insufficient while at least one secondary judge marked it sufficient. These rows are calibration examples. They should be reviewed before increasing context penalties or treating MiMo context labels as clean evidence-mask supervision.

13 Offline Selector / Bandit-Style Baselines

The selector baselines choose among logged candidates only. They do not replace the runtime adaptive heuristic. The main policies are:

- `fixed`: choose one frequent signature; low coverage.
- `cheapest`: choose the lowest resource-cost action.
- `best_average`: choose the exact action signature with highest train mean reward.

- `family_smoothed_best_average`: back off from exact signature to retrieval-context family to context policy.
- `shrinkage_smoothed_best_average`: shrink exact/family/context means toward broader means.
- `linear_reward_model`: ridge/linear offline selector over non-leaking action and cost features.
- `smoothed_linear_selector`: add train-only aggregate means/counts to the linear selector.
- `oracle_logged`: logged-candidate upper bound, not deployable.

The six-seed answer-only sweep showed that `best_average` has the highest non-oracle reward when it covers a group, but coverage is 0.911. `shrinkage_smoothed_best_average` is the strongest full-coverage non-oracle selector so far: reward 0.602, quality 0.773, oracle gap 0.070. The learned linear selectors are functional but not yet robustly stronger than simple smoothed baselines.

Policy	Coverage	Reward	Quality	Oracle gap
cheapest	1.000	0.577	0.770	0.094
best average	0.911	0.618	0.794	0.080
shrinkage smoothed	1.000	0.602	0.773	0.070
linear reward model	1.000	0.586	0.774	0.086
smoothed linear	1.000	0.595	0.767	0.076
oracle logged	1.000	0.671	0.834	0.000

The current bottleneck is data and action coverage, not missing algorithmic complexity.

14 KV-Cache Motivation

The analytical KV-cache estimate used in the project is:

$$\text{KV bytes} \approx 2 \times L \times H_{kv} \times D_{head} \times S \times B \times \text{bytes per value}.$$

For GQA/MQA models, H_{kv} should be the number of key-value heads, not the number of attention heads. The estimates exclude model weights, activations, allocator overhead, paged-attention block overhead, and runtime workspaces.

Model	KV GiB @ 4K	KV GiB @ 32K	KV GiB @ 128K
Qwen2.5-0.5B	0.047	0.375	1.500
Qwen2.5-1.5B	0.109	0.875	3.500
Qwen2.5-3B	0.141	1.125	4.500
Qwen2.5-7B	0.219	1.750	7.000
Qwen2.5-14B	0.750	6.000	24.000

BudgetRAG is the pre-inference step: reduce or improve context before the model runs. Future MemAlign-Qwen work can compare full versus compressed context on local Qwen models, then test runtime KV retention using sink, recent, and evidence tokens.

15 Limitations

- The labels are AI feedback, not human annotations.
- The selector is offline logged-candidate evaluation, not online RL.
- Current logged action coverage is still small and retriever diversity is low.
- Current results support context-budget/action selection more strongly than retrieval-strategy selection.
- Context reward is non-default.
- Pairwise RLAIF currently audits and calibrates scalar reward; it does not train a pairwise selector yet.
- No DPO, PPO, GRPO, online RL, or production runtime KV pruning is implemented.
- Web-search actions, if used, are live stress tests and must not be mixed with reproducible BEIR-style benchmark claims.
- Future MiMo 2.5 runs use standard `mimo-v2.5`; previous Pro-labeled rows remain historical experiment provenance.

16 Future Work

The next steps are:

1. Inspect the 6 MiMo-harsh rows and consensus-insufficient examples.
2. Log retrieval-diversity runs with BM25, graph-BM25, and hybrid-RRF, then analyze context-label evidence quality by retriever and policy.
3. Increase query groups to reduce split variance.
4. Train context-label-aware selectors once supervision is larger.
5. Add a pairwise ranker after enough direct pair labels exist.
6. Profile local Qwen full versus compressed contexts, including prefill/decode latency and peak memory.
7. Later, test a runtime KV-pruning proof of concept that retains sink, recent, and evidence tokens.

17 Threats to Validity

17.1 AI-Judge Bias

MiMo, DeepSeek, Groq, and RAGAS are not human annotators. They may disagree on sufficiency, support, abstention quality, or unsupported-claim risk. The multi-judge audit reduces dependence on a single judge, but it does not replace human validation.

17.2 Logged-Candidate Bias

Offline selectors can only choose among actions that were logged. If the matrix lacks a useful retriever, budget, policy, or model, the selector cannot discover it. This is why retriever-diversity logging is more important than adding complex RL at the current stage.

17.3 Small Held-Out Splits

The split rule avoids query leakage, but the eval sets remain small. Seed 42 looked optimistic for some learned selectors; six-seed evaluation gave a more realistic view. Current selector conclusions should be interpreted as early diagnostic signals.

17.4 RAGAS vs MiMo Scale Mismatch

MiMo quality and RAGAS answer relevancy have correlation 0.277. Absolute reward values from RAGAS-only and MiMo-judge runs should not be compared as if they share the same scale. The useful comparison is the policy ordering and the direction of trade-offs.

17.5 Context Reward Calibration

Context labels materially change reward rows and preference counts. However, penalty 1.00 is too aggressive for a default. Penalty 0.25 is a conservative candidate, but the MiMo-harsh rows need qualitative review before context reward is treated as stable supervision.

18 Completed and Planned Evaluations

18.1 HotpotQA Sampled Kaggle Evaluation

HotpotQA is now included as a sampled Kaggle evaluation, not a full benchmark claim. It is valuable because multi-hop QA stresses evidence sufficiency more directly than simple claim verification. The implementation uses cached BM25 retrieval so the 5.23M-document BEIR HotpotQA corpus is indexed once per notebook instead of once per action row.

The clean MiMo sampled runs provide the usable HotpotQA evidence in this report:

- **Full 16-action sampled matrix:** 800/800 MiMo generations succeeded on 50 sampled queries, with 80 MiMo-backed RAGAS samples. Cached BM25 retrieval gives mean recall@10 0.610 and nDCG@10 0.583. The best token-F1 row is `adaptive-heuristic__balanced__16000` at 0.097, while the best RAGAS answer-relevancy row is `adaptive-heuristic__aggressive__4000` at 0.871.
- **Evidence-aware budget curve:** 300/300 generations succeeded for 1k through 32k budgets, with 30 RAGAS samples. Evidence-aware context saturates after 8k in this sample; 8k is the most defensible trade-off row because it improves answer relevancy without retaining more context at 16k/32k.
- **Low-budget curve:** legacy and adaptive rows over 500–3000 characters completed 750/750 generations, with 75 RAGAS samples. This run tests whether the adaptive policy remains usable under aggressive context limits.

Exact match is 0.000 across the main sampled matrix, so token-F1 and judge metrics are the relevant answer-quality signals for this stage. The Groq Qwen3-32B HotpotQA run is kept as a diagnostic because provider rate limits caused 417/800 generation errors; it must not be mixed with the clean MiMo summary.

18.2 vLLM / Vast AI Model-Bench Workflow

The merged model-bench workflow is an operator/profiling path, not a selector-quality benchmark. It is included because deployment planning for BudgetRAG and future local Qwen experiments depends on

knowing whether candidate models can be served reliably on rented GPUs. The workflow benchmarks local or existing OpenAI-compatible vLLM endpoints, records latency, time-to-first-token, tokens per second, hardware samples, and serving configuration fields such as quantization, KV-cache dtype, attention backend, speculative decoding settings, eager/CUDA-graph mode, batch limits, cache limits, and CPU offload.

The current Vast AI scripts target RTX 5060 Ti 16GB hosts with CUDA 13.0 and CUDA 12.9 profiles. Their outputs are written under ignored `runs/model_bench/` directories. These measurements should be used for deployment triage and cost planning; they are not mixed with the RLAIF selector, HotpotQA, or BEIR benchmark claims.

18.3 Retriever Diversity

Current evidence mostly supports context-budget/action selection. To claim retrieval-context allocation, logged actions must include multiple retrieval strategies:

- BM25 baseline;
- graph-BM25 / dictionary graph where applicable;
- hybrid-RRF;
- optional web-search as live stress test only, not mixed with reproducible BEIR claims.

18.4 Larger Logged Query Groups

The action coverage diagnostics show exact-signature sparsity. More query groups and broader logged action families are needed before a learned selector can robustly beat strong smoothed baselines.

19 Implementation Map

Component	Purpose	Main output
CLI layer	Entry points for benchmark, BudgetRAG, and RLAIF commands.	User-facing commands.
Action normalization	Normalize retrieval-context action dimensions.	Stable action signatures.
RLAIF schemas	Schemas for actions, feedback, rewards, and preferences.	Validated JSONL records.
Dataset builder	Convert BudgetRAG `query_results.jsonl` into normalized datasets.	actions and feedback JSONL.
Answer labeler	AI-judge answer labels with resume and JSON repair.	answer-label JSONL.
Context labeler	AI-judge context sufficiency and chunk labels.	context-label JSONL.
Reward builder	Build scalar rewards and preferences with guardrails.	rewards and preferences JSONL.
Query splitter	Deterministic query-level split.	train/eval JSONL and manifest.

Selector policies	Offline selector baselines.	policy artifact and eval summary.
Pairwise audit	Direct pairwise judge audit.	pairwise-label JSONL.
Context ablation runner	Full postprocess pipeline for context labels.	validation, reward deltas, selector sweeps.
Multi-judge aggregator	Aggregate MiMo/DeepSeek/Groq audit labels.	multi-judge report and summary JSON.
Evidence-quality analyzer	Summarize context-label evidence quality by retriever and context policy.	policy-level evidence diagnostics.
KV estimator	Analytical Qwen KV-cache estimates.	KV estimate report.
Model-bench workflow	Benchmark local or existing vLLM endpoints and record serving and hardware metadata.	model-bench manifests, metrics, and summaries.

The corresponding source files are under `src/rag_bench/` for modules and `scripts/` for standalone analysis scripts.

The code-level test suite covers schema validation, builders, reward/preference logic, labeler hardening, split behavior, selector policies, analysis scripts, model-bench helpers, and secret handling. The merged internship branch currently verifies with 267 passing tests.

A Appendix A: Phase 1B–1C Tables

A.1 Phase 1B Smoke Matrix

Policy / budget	Queries	Kept chars	Compression	Token savings	KV savings
char-budget 1000	10	999.7	0.2161	961.0	900.9
char-budget 2000	10	2000.0	0.4324	711.0	666.6
char-budget 4000	10	3906.0	0.8380	234.2	219.6
evidence-aware 1000	10	999.7	0.2162	961.0	900.9
evidence-aware 2000	10	2000.0	0.4324	711.0	666.6
evidence-aware 4000	10	3999.0	0.8647	234.3	219.7
legacy 1000	10	1002.0	0.2167	960.4	900.4
legacy 2000	10	2005.0	0.4336	709.6	665.2
legacy 4000	10	3914.0	0.8396	232.0	217.5
score-density 1000	10	999.7	0.2162	961.0	900.9
score-density 2000	10	2000.0	0.4324	711.0	666.6
score-density 4000	10	3906.0	0.8380	234.2	219.6

A.2 Phase 1C.2 Adaptive Profile Matrix

Profile / budget	Queries	Kept chars	Compression	Token savings	KV savings
aggressive 1000	50	999.8	0.1248	1829	1715
aggressive 2000	50	1360	0.1665	1739	1630
aggressive 4000	50	2080	0.2500	1559	1462
balanced 1000	50	2260	0.2816	1514	1419
balanced 2000	50	2840	0.3540	1369	1283
balanced 4000	50	4000	0.4991	1079	1012

conservative 1000	50	4000	0.4991	1079	1012
conservative 2000	50	4000	0.4991	1079	1012
conservative 4000	50	4000	0.4991	1079	1012
evidence-aware 1000	50	999.8	0.1248	1829	1715
evidence-aware 2000	50	2000	0.2495	1579	1480
evidence-aware 4000	50	4000	0.4991	1079	1012
legacy 1000	50	1002	0.1250	1829	1714
legacy 2000	50	2006	0.2503	1577	1479
legacy 4000	50	4014	0.5009	1075	1008

Balanced-profile diagnostics at matrix budget 1000: normalized score gap min 0.0071, mean 0.2012, max 0.5288; normalized score entropy min 0.9401, mean 0.9861, max 0.9999; score confidence min 0.00000037, mean 0.0043, max 0.0236.

B Appendix B: RLAIF Dataset and Label Tables

B.1 Selector Smoke Build Summary

Metric	Count
RAGAS source action files	64
Completed RAGAS samples	192
Joined action-query rows	192
`rlaif_actions.jsonl` rows	192
`rlaif_feedback.jsonl` rows	192
`rlaif_rewards.jsonl` rows	192
`rlaif_preferences.jsonl` rows	578
Query groups	55
Action signatures	77
Invalid rows	0
Context-policy preferences	289
Retrieval-context preferences	289
Skipped small reward delta	694
Skipped quality guardrail failed	2

B.2 MiMo Answer Label Summary

Metric	Value
Label count	192
Valid JSON	192
Invalid JSON	0
Ambiguous labels	25
Judge errors	1
Scored labels	191
Used MiMo labels in reward	167
Fallback to original feedback	25
MiMo quality vs RAGAS correlation	0.277

Score	N	Mean	Std	Min	Max
overall_quality	191	0.823	0.307	0.000	1.000
answer_correctness	191	0.857	0.313	0.000	1.000
evidence_support	191	0.825	0.340	0.000	1.000
faithfulness	189	0.847	0.317	0.000	1.000
unsupported_claim_penalty	191	0.090	0.280	0.000	1.000

B.3 Full Context Label Validation

Shard	Rows	Clean usable
1–50	50	46
51–121	71	66
122–192	71	65
Total	192	177

Validation metric	Value
Action count	192
Merged labels	192
Missing actions	0
Unknown action ids	0
Duplicate action ids	0
Duplicate conflicts	0
Clean usable labels	177
Ambiguous labels	15
Invalid JSON labels	0
Dropped unknown chunk ids	0

B.4 Context Reward Delta Summary

Penalty	Changed	Negative	Positive	Mean all delta	Mean changed delta
0.25	140	108	32	-0.155	-0.212
0.50	140	108	32	-0.231	-0.316
1.00	140	108	32	-0.383	-0.525

Penalty	Sufficient false	Sufficient true	Missing sufficiency
0.25	64	72	4
0.50	64	72	4
1.00	64	72	4

B.5 Action Coverage Diagnostics

Level	Unique	Singleton rate	Eval group coverage
action_id	192	1.000	0.000
exact_signature	77	0.260	0.911
retrieval_context_family	16	0.000	1.000
context_policy	3	0.000	1.000
retriever	1	0.000	1.000

C Appendix C: Selector, Pairwise, and Multi-Judge Tables

C.1 Answer-Only Six-Seed Selector Sweep

Policy	Coverage	Reward	Quality	Oracle gap
fixed	0.026 ± 0.057	0.829 ± 0.000	1.000 ± 0.000	0.007
cheapest	1.000 ± 0.000	0.577 ± 0.159	0.770 ± 0.152	0.094
best average	0.911 ± 0.050	0.618 ± 0.147	0.794 ± 0.125	0.080
family smoothed	1.000 ± 0.000	0.598 ± 0.138	0.767 ± 0.122	0.074
shrinkage smoothed	1.000 ± 0.000	0.602 ± 0.117	0.773 ± 0.097	0.070
linear reward model	1.000 ± 0.000	0.586 ± 0.153	0.774 ± 0.110	0.086
smoothed linear	1.000 ± 0.000	0.595 ± 0.137	0.767 ± 0.122	0.076
oracle logged	1.000 ± 0.000	0.671 ± 0.134	0.834 ± 0.138	0.000

C.2 Context-Reward Candidate Selector Sweep

Candidate	Policy	Coverage	Reward	Quality	Oracle gap
penalty 0.25	cheapest	1.000	0.451	0.704	0.093
penalty 0.25	best average	0.911	0.478	0.701	0.085
penalty 0.25	shrinkage	1.000	0.449	0.683	0.095
penalty 0.25	smoothed linear	1.000	0.472	0.689	0.072
penalty 0.25	oracle	1.000	0.544	0.755	0.000
penalty 0.50	cheapest	1.000	0.386	0.704	0.104
penalty 0.50	best average	0.911	0.420	0.701	0.091
penalty 0.50	shrinkage	1.000	0.389	0.683	0.100
penalty 0.50	smoothed linear	1.000	0.415	0.689	0.074
penalty 0.50	oracle	1.000	0.489	0.755	0.000
penalty 1.00	cheapest	1.000	0.255	0.704	0.132
penalty 1.00	best average	0.911	0.303	0.701	0.109
penalty 1.00	shrinkage	1.000	0.273	0.689	0.114
penalty 1.00	smoothed linear	1.000	0.329	0.703	0.058
penalty 1.00	oracle	1.000	0.387	0.748	0.000

C.3 Direct Pairwise MiMo-50 Audit

Metric	Value
Pair labels	50

Valid JSON	50
Invalid JSON	0
Ambiguous labels	2
Judge/API errors	2
A wins	41
B wins	7
Agreement over non-ambiguous	0.854
Mean confidence	0.914
Small quality/support delta pairs	38
Cheaper wins when tied	35
Scalar-over-quality disagreements	5

C.4 Targeted Multi-Judge Audit

Judge	Labels	Valid	Ambiguous	Clean sufficiency
MiMo	60	60	0	57
DeepSeek	60	60	10	50
Groq	60	60	11	49

Pair	Compared	Agree	Agreement rate
MiMo vs DeepSeek	47	43	0.915
MiMo vs Groq	46	40	0.870
DeepSeek vs Groq	41	39	0.951

Audit signal	Count
Consensus insufficient cases	51
Majority vote insufficient	55
Majority vote sufficient	5
High-disagreement cases	6
MiMo-harsh cases	6

D Appendix D: KV Estimates, CLI Chronology, and Sources

D.1 Full Qwen KV-Cache Estimate Table

Table D1 keeps the full analytical KV-cache evidence, but presents it as a compact matrix rather than one row per model/sequence pair. Values are GiB for batch size 1 and fp16/bf16 KV values; they exclude weights, activations, allocator overhead, and paged-attention block overhead.

Model	L/KV	1K	2K	4K	8K	16K	32K	128K
Qwen2.5-0.5B	24/2	0.012	0.023	0.047	0.094	0.188	0.375	1.500
Qwen2.5-1.5B	28/2	0.027	0.055	0.109	0.219	0.438	0.875	3.500
Qwen2.5-3B	36/2	0.035	0.070	0.141	0.281	0.562	1.125	4.500
Qwen2.5-7B	28/4	0.055	0.109	0.219	0.438	0.875	1.750	7.000
Qwen2.5-14B	48/8	0.188	0.375	0.750	1.500	3.000	6.000	24.000

The main implication is the linear growth with sequence length: for Qwen2.5-14B, moving from 4K to 32K raises the estimated KV cache from 0.75 GiB to 6.00 GiB before any runtime overhead is counted. This justifies treating context allocation as a first-class resource decision.

D.2 CLI Chronology

```
rag-bench run / scripts/run_budgetrag_matrix.py
scripts/summarize_budgetrag_results.py
rag-bench rlaif-build
rag-bench rlaif-label-answers
rag-bench rlaif-label-contexts
rag-bench rlaif-reward
rag-bench rlaif-split
rag-bench rlaif-train
rag-bench rlaif-eval
rag-bench rlaif-label-pairs
scripts/summarize_rlaif_labels.py
scripts/summarize_rlaif_context_labels.py
scripts/validate_rlaif_context_labels.py
scripts/run_context_reward_ablation_pipeline.py
scripts/aggregate_rlaif_multijudge_audit.py
```

D.3 Committed Source Reports Used

- `docs/reports/phase1b_smoke_results.md`
- `docs/reports/phase1c_adaptive_smoke_results.md`
- `docs/reports/phase1c1_adaptive_validation.md`
- `docs/reports/phase1c2_adaptive_threshold_calibration.md`
- `docs/reports/phase1d_rlaif_selector_smoke.md`
- `docs/reports/phase1d_rlaif_heldout_eval.md`
- `docs/reports/phase1d_rlaif_ai_judge_heldout_eval.md`
- `docs/reports/phase1d_rlaif_pairwise_mimo50.md`
- `docs/reports/phase1d_rlaif_full_context_reward_ablation.md`
- `docs/reports/phase1d_rlaif_multijudge_audit.md`
- `docs/reports/local_qwen_kv_estimates.md`

References

[1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Chris Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Nova DasSarma Mercado, Robert

Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022. https://arxiv.org/abs/2212.08073.

[2] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyi Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024. https://arxiv.org/abs/2406.02069.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. https://doi.org/10.1145/1571941.1572114.

[4] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. https://arxiv.org/abs/2404.16130.

[5] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023. https://arxiv.org/abs/2309.15217.

[6] Huiqiang Jiang, Qian Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023. https://arxiv.org/abs/2310.05736.

[7] Huiqiang Jiang, Qian Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023. https://arxiv.org/abs/2310.06839.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*, 2020. https://arxiv.org/abs/2005.11401.

[9] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. *arXiv preprint arXiv:1003.0146*, 2010. https://arxiv.org/abs/1003.0146.

[10] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023. https://arxiv.org/abs/2310.06201.

[11] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Andrea Locatelli, Hanchen Ye, Tian Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024. https://arxiv.org/abs/2404.14469.

[12] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024. https://arxiv.org/abs/2402.02750.

[13] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023. https://arxiv.org/abs/2305.18290.

[14] Nandan Thakur, Nils Reimers, Andreas Ruckle, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*, 2021. https://arxiv.org/abs/2104.08663.

[15] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023. https://arxiv.org/abs/2309.17453.

[16] Zhenyu Zhang, Ying Sheng, Tian Zhou, Tian Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*, 2023. https://arxiv.org/abs/2306.14048.